

```
"""
```

```
scaffold_example.py
```

Minimal working example: Bayesian middleware + Ollama.

Drop this pattern into your existing agent loop.

Requirements:

```
    pip install requests
```

```
    Ollama running locally on :11434
```

```
"""
```

```
import json
```

```
import requests
```

```
from bayesian_middleware import BayesianMiddleware, ROUTE_CLARIFY
```

```
OLLAMA_URL = "http://localhost:11434/api/chat"
```

```
OLLAMA_MODEL = "mistral" # or qwen2.5, llama3.2, whatever you're running
```

```
def call_ollama(system_prompt: str, user_message: str, history: list[dict]) -> st
```

```
    messages = [{"role": "system", "content": system_prompt}]
```

```
    messages += history
```

```
    messages.append({"role": "user", "content": user_message})
```

```
    resp = requests.post(OLLAMA_URL, json={
```

```
        "model": OLLAMA_MODEL,
```

```
        "messages": messages,
```

```
        "stream": False,
```

```
    })
```

```
    resp.raise_for_status()
```

```
    return resp.json()["message"]["content"]
```

```
def run_agent(user_id: str = "e404"):
```

```
    mw = BayesianMiddleware(user_id=user_id)
```

```
    history = []
```

```
    print(f"\n— Bayesian Scaffold Agent (user: {user_id}) —")
```

```
    print("Type 'quit' to exit, 'snapshot' to see memory state.\n")
```

```
    while True:
```

```
        user_input = input("You: ").strip()
```

```
        if not user_input:
```

```
            continue
```

```

if user_input.lower() == "quit":
    break
if user_input.lower() == "snapshot":
    snap = mw.snapshot()
    print("\n— Memory Snapshot —")
    print(json.dumps(snap, indent=2))
    print()
    continue

# — 1. Pre-process —————
decision = mw.pre_process(user_input)

print(f"\n[scaffold] bias={decision.bias_detected} "
      f"route={decision.route} "
      f"assertiveness={decision.assertiveness:.0%}")

# — 2. Call Ollama —————
response = call_ollama(decision.system_injection, user_input, history)

# — 3. Post-process (updates memory store) —————
outcome = mw.post_process(response, decision)
print(f"[scaffold] outcome={outcome}\n")

print(f"Agent: {response}\n")

# — 4. Update history —————
history.append({"role": "user", "content": user_input})
history.append({"role": "assistant", "content": response})

# Keep history bounded (last 10 exchanges)
if len(history) > 20:
    history = history[-20:]

if __name__ == "__main__":
    run_agent()

```